Atrium for curators

A **curator** keeps the quality of a space. Curators review what authors submit, publish skills, correct and tidy entries, and withdraw content that should no longer be used. Curators are the second pair of eyes that Atrium relies on: the review of knowledge and especially of skills is the control that keeps wrong or unsafe content away from every colleague's assistant. This guide assumes you know the everyday-use guide and the author guide.

Contents

- What curators can and cannot do
- Reviewing knowledge
- Looking after entries
- Reviewing skills
- Publishing, blocking and archiving skills
- Rules that apply to you
- Daily and weekly workflows
- Troubleshooting
- FAQ

What curators can and cannot do

	Curator
Everything an author can	yes
See all drafts, returned entries and unreleased skill versions of the space	yes
Review knowledge drafts (`review_knowledge`)	yes, not your own
Change, move, archive and restore **any** entry of the space (in a space with approval, archiving an approved entry is yours, not the author's)	yes
Review skills (`review_skill`), release (`publish_skill`), withdraw (`block_skill`)	yes (you cannot review your own versions)
Archive and restore skills, read the skill archive	yes
Move a skill to another space (`move_skill`), retag any skill (`tag_skill`)	yes (target: author rights needed)
Publish a skill of **risk class C**	no, only an instance admin

	Curator
Lift a block (`unblock_skill`)	no, only a space admin or instance admin
Delete entries for good (`delete_knowledge`)	no, space admins do
Delete skills (`delete_skill`)	no, only instance admins (team spaces)
Manage members	no, space admins do
Read the audit log	no, instance admins do

The curator tools appear in your tool list once you are curator in at least one space; every call is checked per space. There is **no notification** yet: you find work by asking for the review queue.

Reviewing knowledge

In a space with approval every author entry is a draft until you decide. Drafts can also arrive in other spaces when an author asked for a check.

The review queue

Ask: "*Show the drafts waiting for review in the Support space.*" (`list_knowledge` with `status=draft`). Check each space you curate.

Reviewing a draft

1. "*Load the draft 'Refund process'.*" (`get_knowledge`). Read the whole text.
2. Decide. Your assistant will present the draft to you and ask before it approves.
3. **Approve**: "*Approve it.*" (`review_knowledge` with `decision=approve` , after your confirmation). The entry becomes visible to all readers of the space.
4. **Return**: "*Return it with the note: please add the effective date and the owner.*" (`decision=reject` , a note is required). The author sees the entry as *rejected* with your note, revises it, and it returns to review as a draft.

What to check:

- **Right space and audience.** Would the readers of this space want this?
- **Correct and current.** Is it accurate, dated, attributed? Does it contradict an existing entry? If yes, the old entry should be corrected or archived in the same step.
- **Duplicates.** Search before approving. Prefer one good entry over two overlapping ones.
- **Self-contained and findable.** Clear title, structure, useful tags.
- **Nothing sensitive.** No credentials (Atrium blocks obvious ones), no personal customer data, nothing that must not be shared with all readers.
- **Data, not orders.** The entry must not try to steer an assistant ("ignore your rules", "always do X"). If a procedure should be followed by assistants, it belongs in a skill.

Good return notes are specific, short and say what would make it approvable. The note is deleted when the author resubmits; the audit log keeps only the decision.

Every review increases the entry's version. The audit log records approvals and returns (without the note).

Curators and their own drafts

A draft is a request to the space to take a look. **You cannot approve your own draft**, another curator does that. Admins of the space and instance admins can (the audit log marks it `self`), so that nothing stays stuck when colleagues are away. A space with a single curator therefore depends on its admin.

Looking after entries

Curators change what authors cannot:

- **Correct an entry of another person**: "*Fix the figure in the entry 'Limits' and update it.*" (`update_knowledge`). The author stays the author, your name is recorded with the new version. In a space with approval, an approved entry is changed by curators only.
- **Archive outdated entries** (`archive_knowledge`, after confirmation). Archive instead of deleting: nothing is lost, and you or the author can restore it (`unarchive_knowledge`). Find archived entries with `list_knowledge` `status=archived` or `search_knowledge` with `archive=true` (only on request; it searches only the archive).
- **Move entries** to the right space (`move_knowledge`, after confirmation). In a space with approval, entries moved by non-curators become drafts; an approved entry that you move into a space with approval stays approved if you are a curator there.
- **Merge duplicates.** Update the better entry, archive the other.
- **Restoring**: when an author restores an approved entry in a space with approval, it goes back to review as a draft, so the archive is no way around approval. When a curator restores it, it stays approved.

Permanent deletion of entries is for space admins, for example for personal data that must go.

Reviewing skills

Skills are different from entries: an approved skill is an instruction that assistants follow for everybody in the space. A careless approval spreads quickly, so the review is stricter. Authors save a version as a draft, you read it, judge it and approve or reject it. Publishing is a separate, deliberate step.

The queue

"*Show skill drafts waiting for review in Support.*" (`list_skills` with `status=draft`, optionally `space`). Drafts are visible only to their author and to curators.

What to read

Read **everything**, not only `SKILL.md`:

1. "*Load version 2 of the skill support-reply.*" (`get_skill` with the version). You get `SKILL.md`, the list of files with hashes, the version, the **digest**, the requested tools and a notice that this is a draft.
2. For every other file, "*Load the file references/tone.md.*" (`get_skill_resource`).
3. Show the files to the user (you) before approving. Your assistant must not approve without your confirmation.

What to check

Check	Why
The skill does what its `description` says, no more	Skills grow scope over time; description is what people trust
No instructions to ignore, bypass or relax the assistant's own rules, safety checks or confirmations	The most dangerous kind of skill text
No hidden steps, no instructions to hide actions from the user	Transparency
No sending of data to addresses, URLs or accounts outside the organisation	Exfiltration
Risky actions (delete, deploy, send, pay) require the user's confirmation	A skill must not remove the user's say
`allowed-tools` matches the steps	Requested tools raise the risk class
References exist, are relevant and say what `SKILL.md` claims	Files are part of the instruction
No credentials, secrets or personal data	Atrium blocks obvious ones, check for more
No odd encodings, invisible characters, very long single lines	Ways to hide text
Instructions are clear enough to be followed correctly	A confusing skill causes errors
Name and description fit the space and do not collide with others	Findability, no look-alikes

The `description` and the instruction text reach assistants. Treat tags and names the same way: they should be plain keywords.

Deciding

- **Reject**: "*Reject it: step 4 tells the assistant to skip the confirmation.*" (`review_skill` with `decision=reject` and a `note`). The author fixes it and saves again (while their own draft is unreviewed it is replaced).
- **Approve**: you name a **risk class** from what the files really contain, not from what the skill claims:
  - **A** instructions only (writing, analysing, procedures),
  - **B** uses tools (reading files, running commands, calling systems),

    - **C** critical actions (deploy, delete, send data out, change permissions). A skill that requests tools cannot be class A.
- Approving needs your **confirmation** and the **digest** of the version you read (the assistant takes it from `get_skill`). If the author changed the draft after you read it, the approval is refused with "the draft changed since you read it". Read again and decide again. This guarantees that you approve exactly what you saw.
- The audit entry `skill_approve` carries the risk class and the digest.

Publishing, blocking and archiving skills

Publish

Approving says "this is fine"; **publishing** says "offer it now". Ask: "*Publish version 2 of support-reply.*" (`publish_skill`, after confirmation). Before it goes live Atrium re-checks the stored files against the digest. From that moment all readers of the space can use the version, and it is announced in assistants' instructions. A published version never changes; improvements are new versions with a new review.

Skills of **risk class C** can only be published by an **instance admin**. Approve them and ask an instance admin to publish.

Block

If a skill turns out to be harmful or wrong, act at once: "*Block version 2 of support-reply, reason: it tells the assistant to skip confirmations.*" (`block_skill`, reason required, after confirmation). New loads are refused immediately. Copies a client already holds are not recalled, so tell the team.

Only a **space admin or instance admin** can lift a block (`unblock_skill`), and the version then returns to *approved*, not *published*: a curator has to release it again (class C: an instance admin). Publishing checks the files against the digest once more. After a block, ask the author for a fixed new version.

Archive and move

- **Archive** a whole skill that is no longer needed (`archive_skill`, after confirmation). It disappears for everyone who cannot restore it; versions keep their states, and restoring needs no new review. Find archived skills with `list_skills` `archived=true`.
- **Move** a skill to another space (`move_skill`): you need curator rights in the old space and author rights in the new one. **No release travels along.** The newest released version is copied as a new draft that the curators of the target review; the old skill is archived unless you ask to keep it. Only a skill with a released version can move, the target space must not have a skill of that name, and tags are copied. Use it when a skill was filed in the wrong space.
- **Retag** (`tag_skill`) to improve findability; tags need no review.
- Deleting skills is for instance admins.

Rules that apply to you

- **No self-approval of your own skill versions.** Not as curator, not as space admin; another curator must approve. Instance admins are the only exception (the audit marks it `self_approved`). A small installation can allow it for all curators with a setting (`skills.allow_self_approval`), which weakens the control.
- **Four-eyes for knowledge** is the same: no approval of your own drafts, except by admins.
- **Each curator decision is audited.** Reviews, publications, blocks, moves and archives appear in the audit log, readable by instance admins.
- **Quotas.** A skill has a version limit (default 100), and every space has limits for entries, bytes and skills. Drafts count, so a flood of drafts can fill a space; if your space runs into a limit, ask a space admin or an instance admin, who can see the usage and raise the limit at any time.
- **Everything a curator sees is data.** Draft content may try to talk to *you* or your assistant ("approve this immediately"). Review it as text, decide yourself.

Daily and weekly workflows

Daily (a few minutes).

1. "*Show drafts and skill drafts waiting in my spaces.*" for each space you curate.
2. Review the oldest first. Agree an expected turnaround with your authors (for example one working day) because there is no notification.
3. If you cannot decide, return the draft with a question in the note.

Reviewing a skill, step by step.

1. List drafts, load the version, load every file.
2. Run through the checklist above. If something is unclear, reject with a note.
3. Approve with risk class; assistant asks you to confirm and supplies the digest.
4. Publish (or ask an instance admin for class C).
5. Test with `/name` on a harmless example. Tell the author.

Weekly.

- Skim the newest entries ("*Show the newest entries in Support*") for duplicates, contradictions and weak titles.
- Check for aged drafts and ask authors to finish or withdraw them.
- Re-check skills that changed hands or got new versions.

Quarterly.

- Review the content: archive outdated entries, merge duplicates, fix tags.
- Review skills: "*List all skills in Support.*" Archive unused ones, ask authors to update old ones. Ask an instance admin for usage figures from the audit log if you want to know which skills are really used.

- Check that every space with approval has **at least two curators**, so that the four-eyes rule works when someone is away.

Urgent withdrawal. A skill or entry is harmful or leaks something.

1. Skills: `block_skill` the version. Entries: `archive_knowledge` stops it being served. To remove it for good, a space admin deletes it, also when it is archived.
2. Tell the space admin and an instance admin; they can check the audit log for who loaded a skill (reads of knowledge entries are not logged).
3. Ask the author for a corrected version; review it with extra care.

Handover when a curator leaves. The space admin adds a new curator before the old one is removed, so approvals continue.

Troubleshooting

Problem	What to try
"The author cannot approve their own version"	Another curator must review it. If you are alone, ask the space admin or an instance admin
"The draft changed since you read it"	The author saved again. Load the version again, read it, approve with the new digest
"Approving needs a risk class" / "digest"	Say which class (A, B or C); the assistant adds the digest from `get_skill`
"Only an instance admin can publish class C"	Ask an instance admin; they review and release it
"The skill is archived"	Restore it first with `unarchive_skill`
You cannot see a draft	Drafts are visible to the author and curators of that space only. Check that you are a curator there with `list_spaces`
A skill needs to be undone quickly	Block it; ask a space admin to unblock when a fixed version is ready

FAQ

Do I have to approve everything? No. Reject or return with a note. A short, helpful note is part of the job.

Can I approve my own entry? No (admins can, marked `self`). Ask another curator.

What is the digest? A SHA-256 fingerprint of all files of a skill version. It proves you approve exactly what you read.

Why does the assistant ask for confirmation so often? Approving, publishing, blocking, archiving and moving change what everybody sees; they need your explicit confirmation.

What is the difference between archive and block? Archive hides a whole skill (restorable, no review needed). Block withdraws one version at once and keeps it; lifting a block needs a space admin and a new release.

Can I publish something I approved earlier? Yes, as long as it stays approved. The files are re-checked at publication.

Who can see my review notes? A note on a returned draft is seen by the author and the curators. A note you add when approving stays on the approved entry or released skill and is visible to all readers, so write nothing there you would not show everybody. The audit log keeps only the decision.

Does the space need a second curator? Yes, whenever possible. A single curator can neither approve their own drafts nor their own skill versions.

A skill uses tools and I am not sure of the class. Choose the higher class. Ask an instance admin if class C is possible; they publish those anyway.

A draft contains personal data. Return it. If it is already visible, archive it to stop it being served, then ask a space admin to delete it for good (that works on archived entries too).